

# Wrangling concepts · reduction & transformation · group\_by

Monday, June 1, 2026

 Today: Monday June 1

## Before class

- Perusall Assignment: [DC §7.1–7.7](#)

## Plan for today

- The wrangling *mindset*: start from the table you want
- The three function families: **reduction**, **transformation**, **data verb**
- First two data verbs: `summarize()` and `group_by()`
- The `na.rm` trap
- The usefulness of `n()` in summaries

## Helpful links

- [Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 3](#) · [DC Ch. 7](#) · [dplyr reference](#)

## Where we are

For two weeks we've been **making plots**. But that assumed the data was already in the right shape, with one row per glyph and one column per aesthetic.

### Tip

Real data almost never arrives glyph-ready. This week is about **getting it there**.

Today we will build the **mental model** that every wrangling function hangs off of.

## Reading the pipe

We've learned `|>`. Read it as the word “**then.**”

```
penguins |>           # take penguins, then
  group_by(species) |> #   group it, then
  summarize(n = n())  #   summarize it
```

### **i** Note

The *DC* reading writes the pipe as `%>%`. `|>` and `%>%` behave identically, so read them as the same word. We'll use `|>`.

## Wrangling mindset

### Start from the table you want

*DC* §7.1 gives the single most useful habit in this whole unit:

Before any coding, sketch the **glyph-ready table** you need, then plan the steps to get there in **plain English**.

For “how does average body mass differ by species?”, the table you *wish* you had is:

| species   | avg_mass |
|-----------|----------|
| Adelie    | ...      |
| Chinstrap | ...      |
| Gentoo    | ...      |

Here we have one row per species and one column for the number you want. **The case is a species, not a penguin.** That shift in what one row represents is from `summarize()` and `group_by()`.

### Plan in English first

*DC*'s smoking example plans the wrangling as four English sentences *before* any code:

1. Make an age-group variable from age.
2. Drop people under 20.

3. Split into groups by age-decade and sex.
4. For each group, count people and count smokers; divide.

Each sentence maps to **one operation**. Today we learn the vocabulary for sentences like #3 (*split into groups*) and #4 (*for each group, count / average*).

## A framework for wrangling

### Three kinds of building blocks

DC §7.2 boils all of wrangling down to three shapes of data...

- A **data frame**: rows (cases) and columns (variables)
- A **variable**: one column
- A **scalar**: a single value

and three families of functions, sorted by **what goes in and what comes out**:

| Family                | Input → Output          | Examples                                                                                                                                           |
|-----------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Reduction</b>      | variable → scalar       | <code>mean()</code> , <code>sum()</code> , <code>n()</code> , <code>median()</code> , <code>sd()</code> , <code>n_distinct()</code>                |
| <b>Transformation</b> | variable(s) → variable  | <code>x / y</code> , <code>log10()</code> , <code>round()</code> , <code>ifelse()</code> , <code>&lt;</code> , <code>==</code> , <code>%in%</code> |
| <b>Data verb</b>      | data frame → data frame | <code>summarize()</code> , <code>group_by()</code> , <code>filter()</code> , <code>mutate()</code> , ...                                           |

### Reduction functions: many → one

A **reduction** takes a whole column and crushes it down to a single number (DC §7.3).

```
mean(penguins$body_mass_g, na.rm = TRUE) # one number out
```

```
[1] 4201.754
```

We've used many of these before: `mean()`, `sum()`, `median()`, `sd()`, `min()`, `max()`. Here are two new ones:

- `n()` — how many cases (takes **no** arguments)
- `n_distinct()` — how many *different* values

### Note

`n()` doesn't look at any column, it simply counts rows. It only works *inside* a data verb like `summarize()`, not on its own.

## Transformation functions: column to column

A **transformation** runs once *per row* and hands back a column the same length as the input (*DC* §7.4).

```
head(penguins$body_mass_g / 1000) # grams to kg, one value per row
```

```
[1] 3.75 3.80 3.25 NA 3.45 3.65
```

Comparisons are transformations too, they return a column of TRUE/FALSE:

```
head(penguins$body_mass_g > 4000)
```

```
[1] FALSE FALSE FALSE NA FALSE FALSE
```

### Tip

**Reduction vs. transformation in one question:** does it give back *one* number, or *one per row*? `mean()` to one. `round()` to one per row.

## Data verbs: data frame to data frame

A **data verb** takes a whole data frame and returns a whole (new) data frame (*DC* §7.5). Here are three rules every dplyr verb obeys:

1. The **first argument is always a data frame**.
2. The other arguments name **columns**, unquoted.
3. The **output is always a new data frame**; the input is never modified.

```
penguins |> summarize(avg = mean(body_mass_g, na.rm = TRUE))  
# penguins itself is unchanged; you get a NEW data frame back
```

### ⚠ Warning

Because verbs never modify their input, `penguins |> summarize(...)` on its own just **prints**. To keep a result, assign it: `out <- penguins |> summarize(...)`.

## Each step = one verb + helpers

Each step in data wrangling involves a **data verb** and one or more **reduction or transformation functions**.

The verb is the *action on the table*; the reduction/transformation is the *detail of what to compute*.

```
penguins |>
  summarize(avg_mass = mean(body_mass_g, na.rm = TRUE))
#           data verb      reduction function
```

## summarize()

### summarize(): collapse a table to a summary

`summarize()` turns many rows into **one row**, using reduction functions.

```
penguins |>
  summarize(
    n      = n(),
    avg_mass = mean(body_mass_g, na.rm = TRUE),
    max_mass = max(body_mass_g, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 3
   n avg_mass max_mass
<int>   <dbl>   <int>
1   344   4202.    6300
```

Things to notice:

- Each **named argument** becomes a **column** in the output.
- The output is a **data frame** — one row here, but still a data frame.

## The `na.rm` trap

Watch what happens without `na.rm = TRUE`:

```
penguins |>
  summarize(avg_mass = mean(body_mass_g))
```

```
# A tibble: 1 x 1
  avg_mass
  <dbl>
1      NA
```

One missing value anywhere in the column makes the **whole answer** NA. The reduction can't average around a hole it doesn't know how to fill.

### Warning

This is the AI failure we dissected on Friday: code that runs, looks fine, and quietly returns NA. The fix is `na.rm = TRUE`, but the *skill* is noticing the NA in the first place.

## `group_by()`

### `group_by()`: summarize, but per group

`group_by()` doesn't compute anything itself. It **tags** the data frame so the *next* verb runs once per group.

```
penguins |>
  group_by(species) |>
  summarize(
    n      = n(),
    avg_mass = mean(body_mass_g, na.rm = TRUE)
  )
```

```
# A tibble: 3 x 3
  species      n avg_mass
  <fct>    <int>   <dbl>
1 Adelie   152   3701.
2 Chinstrap  68   3733.
3 Gentoo  124   5076.
```

Same `summarize()` as before, now **one row per species** instead of one row total.

**i** Note

`group_by()` should (almost) always be followed by another verb. On its own it just attaches an invisible label, the work happens in the verb after it.

## Grouping by more than one variable

List several columns to get one row per **combination**:

```
penguins |>
  group_by(species, sex) |>
  summarize(
    n      = n(),
    avg_mass = mean(body_mass_g, na.rm = TRUE)
  )
```

```
# A tibble: 8 x 4
# Groups:   species [3]
  species sex      n avg_mass
  <fct>   <fct> <int>   <dbl>
1 Adelie female   73   3369.
2 Adelie male     73   4043.
3 Adelie <NA>      6   3540
4 Chinstrap female  34   3527.
5 Chinstrap male    34   3939.
6 Gentoo  female   58   4680.
7 Gentoo  male    61   5485.
8 Gentoo  <NA>     5   4588.
```

Two things worth a pause:

- There's an NA row for `sex` indicating some penguins have missing sex. `group_by()` keeps them as their own group. (Dropping them needs `filter()`.)
- You may see a message about the output being “*regrouped*”. This is harmless for us, and we can add `.groups = "drop"` to silence it.

## How group\_by() changes the case

Wrangling changes **what one row means**.

Table 1: group\_by() resets the meaning of a case.

| Pipeline                                        | One row =               |
|-------------------------------------------------|-------------------------|
| penguins                                        | one penguin             |
| ...  > summarize(n = n())                       | the whole dataset       |
| ...  > group_by(species)  > summarize(...)      | one species             |
| ...  > group_by(species, sex)  > summarize(...) | one species × sex combo |

### Tip

Before you run a `group_by() |> summarize()`, predict **how many rows** the result has. It's the number of distinct group combinations. If your prediction is wrong, you've learned something about your data.

## Trust, but verify

### Always include an n()

Average mass by the finest grouping:

```
penguins |>
  group_by(species, island, sex) |>
  summarize(
    n      = n(),
    avg_mass = mean(body_mass_g, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 13 x 5
  species island sex      n avg_mass
  <fct>   <fct> <fct> <int>   <dbl>
1 Adelie Biscoe female    22  3369.
2 Adelie Biscoe male     22  4050
3 Adelie Dream  female    27  3344.
```



|    |           |           |        |    |       |
|----|-----------|-----------|--------|----|-------|
| 4  | Adelie    | Dream     | male   | 28 | 4046. |
| 5  | Adelie    | Dream     | <NA>   | 1  | 2975  |
| 6  | Adelie    | Torgersen | female | 24 | 3396. |
| 7  | Adelie    | Torgersen | male   | 23 | 4035. |
| 8  | Adelie    | Torgersen | <NA>   | 5  | 3681. |
| 9  | Chinstrap | Dream     | female | 34 | 3527. |
| 10 | Chinstrap | Dream     | male   | 34 | 3939. |
| 11 | Gentoo    | Biscoe    | female | 58 | 4680. |
| 12 | Gentoo    | Biscoe    | male   | 61 | 5485. |
| 13 | Gentoo    | Biscoe    | <NA>   | 5  | 4588. |

Some groups have a handful of birds, others have dozens. **A mean from a group of 3 is not the same kind of fact as a mean from a group of 60.**

## Good habit

### Warning

Whenever you compute a group summary, **add `n = n()`**. A surprising-looking average is very often just a tiny group. Without the count, you can't tell a real effect from noise.

This is also a verification move, exactly like Friday: the number that *looks* like an answer might be resting on almost no data.

## Tomorrow

### What's next

Today: the framework + `summarize()` + `group_by()`. That's already enough to answer a lot of “how many / what's the average, by group” questions.

Tomorrow (*DC* Ch. 10) we add the four verbs:

- `filter()` — keep some **rows** (e.g. drop the NA sexes)
- `select()` — keep some **columns**
- `mutate()` — add a **new column** (this is where transformations live)
- `arrange()` — **reorder** rows

With the two we covered today, these handle “take a messy CSV and produce a useful summary.”

## What we covered today

- **The mindset:** sketch the glyph-ready table, plan in English, one sentence per operation
- **The framework:** reduction (variable→scalar), transformation (variable→variable), data verb (frame→frame)
- **summarize():** many rows to one; named args become columns; output is a data frame
- **na.rm = TRUE:** one NA poisons a whole reduction; spotting it is the skill
- **group\_by():** tags the frame so the next verb runs per group; changes what a “case” means
- **n():** include it in every summary so you never trust a mean from three rows

## Activity

### Activity: summarising the penguins

Roughly 25 minutes. Individually or with a neighbor.

- [Activity Canvas Link](#)

You’ll classify functions into the three families, build up `summarize()` and `group_by()` pipelines on `penguins`, deliberately trip the `na.rm` trap, and practice predicting the **shape** of a grouped result before you run it.

**To turn in:** save your `.qmd` and rendered HTML.

## Wrap-up & looking ahead

**Tomorrow (Tuesday):** the row and column verbs, `filter`, `select`, `mutate`, `arrange`, the rest of the dplyr toolkit.

### Upcoming Deadlines

- **Tue 6/2, before class:** [DC §10.1–10.6](#)
- **Thu 6/4, 11:59 p.m.:** [Project: Team formation](#)
- **Fri 6/5, 11:59 p.m.:** [Homework 3: Data Wrangling & Reshaping](#)

### Reach out

Stuck? Office hours, or email me ([cgc5478@psu.edu](mailto:cgc5478@psu.edu)) or Xinyue ([xpw5228@psu.edu](mailto:xpw5228@psu.edu)).

### Reference

[Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 3](#) · [DC Ch. 7](#) · [dplyr reference](#)